Graphic Era deemed to be University DEHRADUN

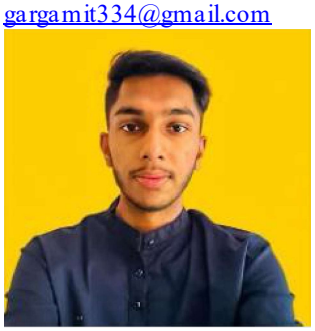
PROJECT AND TEAM INFORMATION

Project Title

TinyLang: LL(1) Based Static Complexity & Optimization Web Compiler

Student/Team Information

Team Name: Team #	EduSquad CD-VI-T062
Team member 1 (Team Lead) (Last Name, name: student ID: email, picture):	Juyal, Ayush – 230213859 ayushjuyal.aj.7@gmail.com
Team member 2 (Last Name, name: student ID: email, picture):	Pradhan, Avish - 230213832, avishpradhan094@gmail.com

Team member 3 (Last Name, name: student ID: email, picture):	Garg,Nandan – 230213857 gargamit334@gmail.com 

PROPOSAL DESCRIPTION

Motivation

Traditional compiler projects focus mainly on lexical and syntax analysis. While these components ensure program correctness, they do not analyze algorithm efficiency or suggest performance improvements. Additionally, many student projects rely on parser generators without demonstrating theoretical parsing concepts such as LL(1) predictive parsing.
This project aims to build TinyLang, a smart compiler that integrates:

- LL(1) Predictive Parsing
- AST Construction
- Static Time Complexity Estimation
- Performance Hotspot Detection
- Static Code Optimization Suggestions
- Flask-Based Web Interface

By combining theoretical compiler concepts with practical performance analysis, this system enhances understanding of compiler phases and algorithm efficiency, making it both academically strong and practically useful.

State of the Art / Current solution

Modern compilers such as GCC and Clang perform syntax checking and low-level optimizations but do not explicitly provide Big-O time complexity estimation or educational hotspot analysis.
Performance analysis tools (profilers) require program execution, and static analyzers provide limited warnings. Traditional student compiler projects typically implement lexical and syntax analysis only.
There is a gap in educational tools that combine:

- Formal LL(1) parsing
- Static complexity estimation
- Hotspot identification
- Optimization suggestions
- Web-based interaction

TinyLang addresses this gap by integrating all these features into one structured system.

Project Goals

General Goals:

1. Design TinyLang grammar compatible with LL(1).
2. Compute FIRST and FOLLOW sets.
3. Construct LL(1) parsing table.
4. Implement predictive parser.
5. Construct Abstract Syntax Tree (AST).
6. Implement static complexity analyzer.
7. Detect performance hotspots.
8. Implement static optimization module.
9. Integrate Flask backend.

Milestones:

- Milestone 1: Grammar Design & Lexer Implementation
- Milestone 2: LL(1) Parsing Table Construction
- Milestone 3: AST Builder
- Milestone 4: Complexity Analyzer
- Milestone 5: Hotspot Detection
- Milestone 6: Code Optimization Module
- Milestone 7: Flask Integration
- Milestone 8: Testing & Documentation

Project Approach

The project follows a modular compiler architecture integrated with a web backend.

Processing Flow:

User Code (Web Interface)
→ Flask Backend
→ Lexical Analyzer (Flex)
→ LL(1) Predictive Parser
→ AST Construction
→ Complexity Analyzer
→ Hotspot Detection
→ Code Optimization Module

→ JSON Output
→ Web Display

Technologies Used:

- C (Core Compiler Implementation)
- Flex (Lexical Analysis)
- Custom LL(1) Parser
- Flask (Backend Framework)
- JSON (Structured Output)
- HTML/CSS/JS (Frontend)

Key Algorithms:

- LL(1) Predictive Parsing
- FIRST & FOLLOW Computation
- DFS Traversal on AST
- Loop Nesting Depth Analysis
- Symbolic Bound Evaluation
- Static Optimization Heuristics

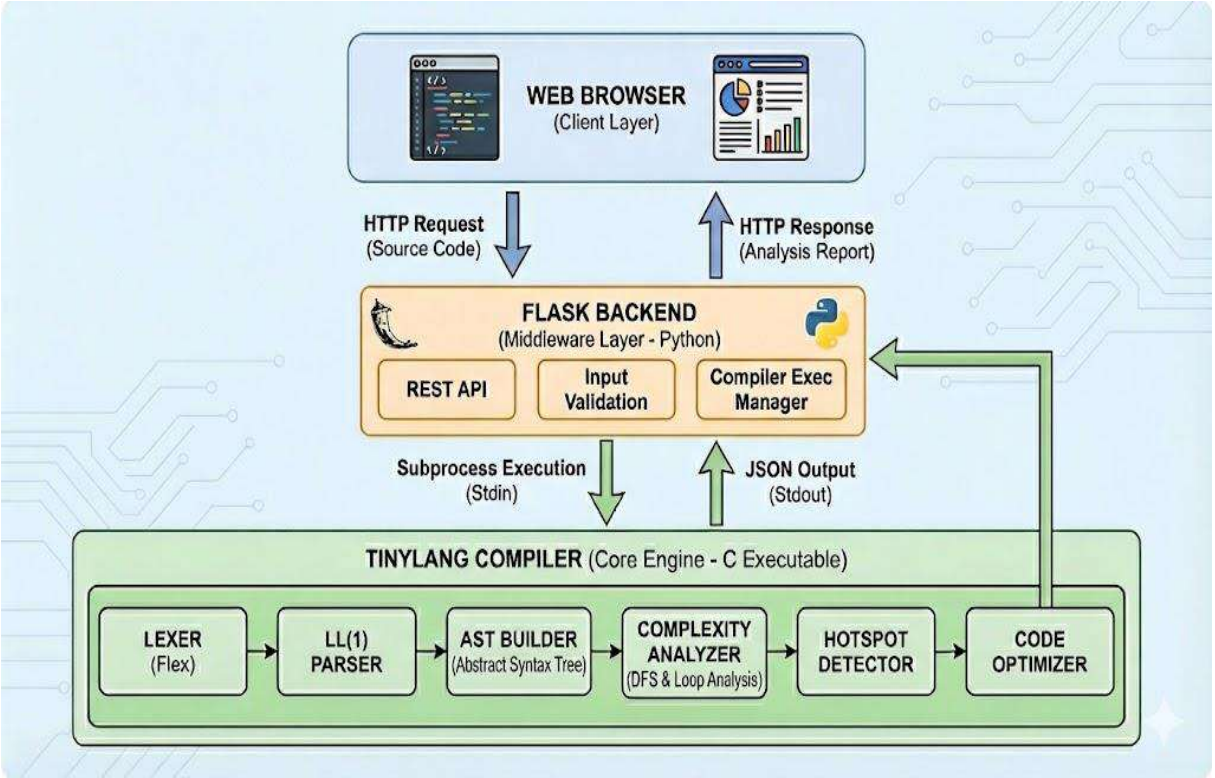
System Architecture (High Level Diagram)

The **Compiler & Analysis Layer** is implemented in C and consists of multiple modules:

- Lexer (Flex) for tokenization
- LL(1) Predictive Parser for syntax analysis
- AST Builder for structural representation
- Complexity Analyzer for Big-O estimation
- Hotspot Detector for identifying performance-critical regions
- Code Optimization Module for static improvement suggestions

The overall data flow is:

User Input → Flask Backend → Lexer → LL(1) Parser → AST → Complexity Analyzer → Hotspot Detector → Optimizer → JSON Output → Web Display



Project Outcome / Deliverables

At completion, the project will deliver:

1. Fully functional TinyLang compiler.
2. Working LL(1) predictive parser with parsing table.
3. AST construction module.
4. Static Big-O complexity estimator.
5. Performance hotspot detector.
6. Static code optimization suggestion system.
7. Flask-based web interface.
8. Structured JSON analysis output.
9. Complete project documentation.

Example Output:

Estimated Time Complexity: $O(n^2)$
Maximum Loop Depth: 2
Hotspot: Nested loop at lines 4–8

Optimization Suggestions:
- Reduce nested loop depth
- Apply loop invariant code motion
- Consider strength reduction

Assumptions

- Input programs follow TinyLang LL(1) grammar
- Worst-case complexity estimation is acceptable.
- Optimization suggestions are static (educational level).
- Recursion analysis may be limited.
- Flask server runs locally or on institutional environment.

References

- Aho, Lam, Sethi, Ullman – *Compilers: Principles, Techniques, and Tools* – https://www.pearson.com/en-us/subject-catalog/p/compilers-principles-techniques-and-tools/P200000003479
- Cormen et al. – *Introduction to Algorithms* – https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/
- Flex Documentation – https://westes.github.io/flex/manual/
- GNU Bison Documentation – https://www.gnu.org/software/bison/manual/
- Flask Official Documentation – https://flask.palletsprojects.com/